



CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL

UNIVERSIDAD ESTATAL DE GUAYAQUIL

FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS

CARRERA DE CIENCIAS DE DATOS E INTELIGENCIA ARTIFICIAL

CURSO:

CDDEIA-ELNO-4-2

**AGENTE CONVERSACIONAL PARA ADMISIÓN Y NIVELACIÓN
DE LA UG**

ESTUDIANTES:

QUINDE PONCE ALEXIS ERNESTO

PILOZO RODRÍGUEZ HERNÁN ANDRÉS

ASIGNATURA:

PROCESAMIENTO DEL LENGUAJE NATURAL

DOCENTE:

ING. CUENCA ORTEGA ANGEL

PERÍODO LECTIVO

Ciclo I 2026 – 2027

Contenido

Introducción al problema abordado	3
Diseño de intenciones y entidades.....	3
Ejemplo de intenciones (data/intenciones.json)	4
Entidades del dominio (entidades.py)	4
Arquitectura del agente.....	5
Funcionamiento.....	6
Justificación	7
a) TF-IDF frente a bolsa de palabras (conteos simples).....	7
b) Similitud coseno frente a distancia euclidiana.....	7
c) Preprocesamiento antes de vectorizar	8
d) Umbral de confianza y respuesta de fallback	8
e) Datos (JSON) separados del código.....	9
f) Manejo de errores con try/except	9
Conclusiones	10

Introducción al problema abordado

Cada periodo de admisión, la Universidad de Guayaquil recibe una gran cantidad de preguntas repetidas por parte de los aspirantes: requisitos para inscribirse, fechas de registro y matrícula, cómo crear una cuenta, cómo se aprueban las materias del curso de nivelación, disponibilidad de cupos, aceptación de carrera, entre otras.

Este proyecto implementa un agente conversacional (chatbot) que responde, en español, preguntas frecuentes sobre admisión y nivelación de la UG, usando únicamente técnicas clásicas de Procesamiento de Lenguaje Natural. No se utilizan modelos de deep learning, transformers ni generación abierta de lenguaje, de acuerdo con el alcance definido para el trabajo.

Además de los temas de admisión y nivelación, se agregaron algunas intenciones con información general de la propia Universidad de Guayaquil (historia, misión y visión, facultades, y redes sociales oficiales), obtenida directamente del sitio institucional www.ug.edu.ec, para que el agente se sienta más completo como asistente virtual del sitio de admisiones.

Diseño de intenciones y entidades

El agente maneja ciertas intenciones, definidas en el archivo `data/intenciones.json`, cada una con un conjunto de utterances (frases de ejemplo de cómo un usuario podría preguntar eso) y una o varias respuestas predefinidas. Se agruparon en tres bloques:

- **Conversacionales:** saludo, despedida, agradecimiento, mostrar_ayuda e identidad_bot.
- **Núcleo de admisión y nivelación:** preguntar_requisitos_admision, explicar_creacion_cuenta, consultar_fechas_registro, consultar_cronograma_matricula, preguntar_curso_nivelacion, consultar_aprobacion_asignaturas, consultar_calificaciones, consultar_control_asistencia, consultar_cupos, consultar_aceptacion_carrera, preguntar_documentos_necesarios y consultar_canales_contacto, entre otros.
- **Información general de la UG:** info_general_universidad, historia_universidad, mision_vision_universidad, facultades_ug, ubicacion_campus, redes_sociales_universidad y plataformas_en_linea.

Ejemplo de intenciones (data/intenciones.json)

```
PROBLEMAS    SALIDA    CONSOLA DE DEPURACIÓN    TERMINAL    PUERTOS
```

```
PS D:\chatbot_corregido(ultimo)\chatbot> & C:\Users\YSTEL\AppData\Local\Programs\Python\Python314\python.exe "d:/chatbot_corregido(ultimo)/chatbot/src/chatbot.py"
```

```
Tú: hola
```

```
Bot: Hola, soy Alfred un asistente virtual. ¿En que puedo ayudarte?
```

```
Intent: saludo
```

```
Similitud: 1.0
```

```
Entidades: {'fechas': [], 'cedula': [], 'carreras': [], 'terminos_dominio': []}
```

```
Tú: cual es la historia de la UG
```

```
Bot: La Universidad de Guayaquil tiene su origen el 1 de diciembre de 1867, con la primera sesion de la Junta Universitaria del Guayas. Se inauguro formalmente el 9 de octubre de 1883 por Pedro Carbo, y desde 1885 se la conoce oficialmente como Universidad de Guayaquil. Puedes revisar la línea de tiempo completa en https://www.ug.edu.ec/historia/.
```

```
Intent: historia_universidad
```

```
Similitud: 0.779
```

```
Entidades: {'fechas': [], 'cedula': [], 'carreras': [], 'terminos_dominio': []}
```

```
Tú: chao
```

```
Bot: Espero que logres lo que buscas.
```

```
Intent: despedida
```

```
Similitud: 1.0
```

```
Entidades: {'fechas': [], 'cedula': [], 'carreras': [], 'terminos_dominio': []}
```

```
Tú:
```

En este ejemplo realizamos tres preguntas (intenciones) las cuales fueron:

- Saludo
- Historia de la UG
- Despedida

Entidades del dominio (entidades.py)

Además de la intención, el agente extrae entidades mediante expresiones regulares (RF-05), definidas en `src/entidades.py`:

- **Fechas:** formato numérico (dd/mm/aaaa) y en texto ("10 de julio de 2026").

```
38 # Fechas: dd/mm/yyyy, dd-mm-yyyy, "10 de julio de 2026", "10 de julio"
39 _PATRON_FECHA_NUMERICA = r"\b\d{1,2}[/-]\d{1,2}(?:[/-]\d{2,4})?\b"
40 MESES = (
41     "enero|febrero|marzo|abril|mayo|junio|julio|agosto|"
42     "septiembre|setiembre|octubre|noviembre|diciembre"
43 )
44 PATRON_FECHA_TEXTO = rf"\b\d{{1,2}}\s+de\s+(?:{MESES})(?:\s+de\s+d{{4}})?\b"
```

- **Cédula:** 10 dígitos consecutivos.

```
46 # Cedula ecuatoriana: 10 digitos consecutivos
47 PATRON CEDULA = r"\b\d{10}\b"
```

- **Carreras:** lista cerrada de carreras de la UG (ingeniería en sistemas, medicina, derecho, psicología, etc.), detectadas por coincidencia de texto.

```
9 # Lista de carreras UG frecuentemente mencionadas
10 _CARRERAS = [
11     "ciencia de datos e inteligencia artificial",
12     "ingenieria en sistemas",
13     "ingenieria civil",
14     "medicina",
15     "derecho",
16     "psicologia",
17     "administracion de empresas",
18     "contaduria publica",
19     "arquitectura",
20     "enfermeria",
21     "odontologia",
22 ]
```

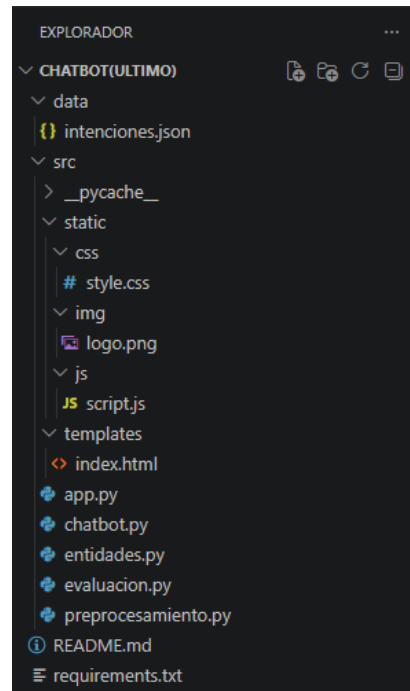
- **Términos de dominio:** expresiones propias del proceso, como "curso de nivelacion", "cupo aceptado", "cronograma de registro".

```
24 # Terminos especificos del proceso (dominio) a detectar tal cual aparecen
25 _TERMINOS_DOMINIO = [
26     "curso de nivelacion",
27     "cupo aceptado",
28     "cupo",
29     "matricula ordinaria",
30     "matricula",
31     "cronograma de registro",
32     "cronograma",
33     "cuenta de aspirante",
34     "creacion de cuenta",
35 ]
```

Arquitectura del agente

El proyecto sigue una arquitectura simple, dividida en módulos con una responsabilidad concreta cada uno:

- **data/intenciones.json** — intenciones, utterances, respuestas y mensajes de fallback (los datos, separados del código).
- **src/preprocesamiento.py** — limpieza y normalización del texto: minúsculas, eliminación de tildes y signos de puntuación, corrección de jerga/errores tipográficos comunes, tokenización, eliminación de stopwords.
- **src/entidades.py** — extracción de entidades del dominio mediante expresiones regulares.
- **src/chatbot.py** — clase AgenteConversacional: carga los datos, construye el vectorizador, calcula la similitud coseno entre la consulta y el corpus, aplica el umbral de confianza y arma la respuesta final.
- **src/app.py** — servidor Flask que expone el agente como interfaz web.
- **src/evaluacion.py** — script independiente que prueba el agente con un conjunto de consultas y calcula métricas.



Funcionamiento

El flujo de una consulta es el siguiente: el usuario escribe un mensaje (por consola o por la interfaz web) → preprocesar() limpia y normaliza el texto → el vectorizador transforma la consulta a un vector → cosine_similarity() la compara contra todas las utterances del corpus → se toma la intención de mayor similitud → si supera el umbral de confianza (0.50), devuelve una respuesta asociada; de lo contrario, responde con un mensaje de fallback. Además, si detecta entidades relevantes, agrega información adicional a la respuesta.

```
PS D:\ChatbotProyecto\chatbot(ultimo)> & C:\Users\YSTEL\AppData\Local\Programs\Python\Python314\python.exe "d:/ChatbotProyecto/chatbot(ultimo)/src/chatbot.py"
Tú: hola

Bot: Hola, soy Alfred un asistente virtual. ¿En que puedo ayudarte?
Intent: saludo
Similitud: 1.0
Entidades: {'fechas': [], 'cedula': [], 'carreras': [], 'terminos_dominio': []}

Tú: que documentos necesito para inscribirme

Bot: Tu duda sobre admision seguramente es sobre los requisitos generales de admision, puedes encontrar esa informacion publicada en el portal oficial https://admision.ug.edu.ec/admision/.
Debes crear una cuenta de aspirante y presentar la documentacion solicitada en el cronograma vigente.
Tambien puedes consultar tu duda en las paginas oficiales de la UG que puedes encontrar solo buscando en cualquier red social contenido sobre la UG .
Intent: preguntar_requisitos_admision
Similitud: 0.618
Entidades: {'fechas': [], 'cedula': [], 'carreras': [], 'terminos_dominio': []}

Tú: como creo una cuenta en admision UG

Bot: La creacion de cuentas se gestiona exclusivamente en el Portal Oficial de Admision UG (https://admision.ug.edu.ec/admision/). Ingresa tus datos personales y numero de cedula para generar tu cuenta de aspirante.
Intent: explicar_creacion_cuenta
Similitud: 0.685
Entidades: {'fechas': [], 'cedula': [], 'carreras': [], 'terminos_dominio': []}

Tú: 
```

Justificación

a) TF-IDF frente a bolsa de palabras (conteos simples)

En el proyecto, TfidfVectorizer(ngram_range=(1,2), sublinear_tf=True) no solo cuenta cuántas veces aparece una palabra, sino que multiplica esa frecuencia por el factor IDF (Inverse Document Frequency), que castiga a las palabras que aparecen en casi todas las utterances (por ejemplo "universidad" o "admisión", que salen en muchas intenciones) y da más peso a las palabras que son propias de pocas intenciones (por ejemplo "cupo", "nivelación", "cedula"). Con una simple bolsa de palabras (conteos), esas palabras poco informativas pesarían igual que las palabras realmente distintivas, y el modelo tendría más dificultad para separar intenciones que comparten vocabulario general.

```
Tú: como llego al apartado para subir documentos

Bot: Claro para llegar al apartado para subir tus documentos una vez ya iniciado sesión en SIUG sigue
estos pasos:
1. Te saldrán anuncios de la universidad, puedes cerrarlos
2. En la parte superior izquierda en 'Académico' debes dar click
3. Entra en 'Estudiante' y luego a datos personales
4. Listo, entre ahí están todas las opciones para empezar a subir y llenar lo necesario.
Intent: apartado_SIUG
Similitud: 0.821
Entidades: {'fechas': [], 'cedula': [], 'carreras': [], 'terminos_dominio': []}

Tú: 
```

b) Similitud coseno frente a distancia euclidiana

Para comparar la consulta del usuario contra las utterances se usa cosine_similarity(vector_consulta, matriz_tfidf), no distancia euclidiana. La similitud coseno mide el ángulo entre los vectores, no su magnitud, por lo que dos frases con el mismo tema pero distinta longitud (por ejemplo "cupo" frente a "todavía hay cupos disponibles para mi carrera") pueden seguir teniendo una similitud alta si usan palabras parecidas. La distancia euclidiana, en cambio, sí depende del tamaño de los vectores: una frase larga tendría naturalmente una norma mayor y aparecería más "lejos" aunque hable exactamente del mismo tema. En otras palabras, la normalización de longitud implícita en el coseno evita que el largo de la frase distorsione la comparación.

```
Tú: como llego al apartado para subir documentos

Bot: Claro para llegar al apartado para subir tus documentos una vez ya iniciado sesión en SIUG sigue
estos pasos:
1. Te saldrán anuncios de la universidad, puedes cerrarlos
2. En la parte superior izquierda en 'Académico' debes dar click
3. Entra en 'Estudiante' y luego a datos personales
4. Listo, entre ahí están todas las opciones para empezar a subir y llenar lo necesario.
Intent: apartado_SIUG
Similitud: 0.821
Entidades: {'fechas': [], 'cedula': [], 'carreras': [], 'terminos_dominio': []}

Tú: 
```

```
{
  "intent": "apartado_SIUG",
  "utterances": [
    "Puedes decirme como llego hasta el apartado para subir los documentos?",
    "apartado para subir",
    "apartado"
  ],
  "respuestas": [
    "Claro para llegar al apartado para subir tus documentos una vez ya iniciado sesión en SIUG sigue estos"
  ]
}
```

c) Preprocesamiento antes de vectorizar

La función `preprocesar()` de `preprocesamiento.py` aplica, en orden: limpieza de texto y tildes, tokenización, normalización de jerga/errores comunes, eliminación de stopwords y stemming. Este orden es importante porque reduce la variabilidad del vocabulario antes de contar palabras. Un ejemplo propio del proyecto: la palabra "admisión" puede escribirse como "admision", "admicion" o "amision" en consultas reales; el diccionario `_JERGA_Y_TYPOS` normaliza esas variantes a "admision" antes de continuar el pipeline. De la misma forma, el stemming reduce palabras como "matricularme", "matricula" y "matriculación" a una raíz común, de modo que el vectorizador las trata como la misma palabra en vez de tratarlas como tres términos distintos sin relación entre sí. Sin este paso, muchas variantes naturales del español (con o sin errores de tecleo) no coincidirían con las utterances del corpus y la similitud coseno resultante sería artificialmente baja.

```
Tú: cuales son los requisitos para la amision

Bot: Tu duda sobre admision seguramente es sobre los requisitos generales de admision, puedes encontrar esa informacion publicada en el portal oficial https://admision.ug.edu.ec/admision/. Debes crear una cuenta de aspirante y presentar la documentacion solicitada en el cronograma vigente. Tambien puedes consultar tu duda en las paginas oficiales de la UG que puedes encontrar solo buscando en cualquier red social contenido sobre la UG.
Intent: preguntar_requisitos_admision
Similitud: 0.65
Entidades: {'fechas': [], 'cedula': [], 'carreras': [], 'terminos_dominio': []}
```

d) Umbral de confianza y respuesta de fallback

La constante `UMBRAL_CONFIANZA_DEFECTO = 0.50` en `chatbot.py` define el punto mínimo de similitud que debe alcanzar la mejor coincidencia para que el agente responda con una intención; si no lo alcanza, `detectar_intencion()` devuelve `None` y `responder()` usa una respuesta de fallback en su lugar. El valor 0.50 se eligió de forma empírica corriendo `evaluacion.py` varias veces: con umbrales más bajos (por ejemplo 0.3) el agente empezaba a responder con seguridad a preguntas totalmente fuera de tema; con umbrales más altos (por ejemplo 0.7) varias preguntas válidas, pero escritas de forma distinta a las utterances de entrenamiento, terminaban cayendo en fallback sin necesidad. Si no existiera ningún umbral, el agente siempre devolvería la intención de mayor similitud aunque esta fuera prácticamente 0 (por ejemplo, ante una consulta como "asdkjaskjd" o una pregunta totalmente fuera de alcance), dando respuestas incorrectas con apariencia de certeza en vez de admitir que no entendió la consulta.

```
Tú: cuanto pesa un elefante

Bot: La universidad de Guayaquil cuenta con:
-> 17 Facultades
-> 59 carreras vigentes
-> Modalidad: Presencial, Híbrida y en línea
Puedes consultar toda la informacion sobre las carreras en: https://admision.ug.edu.ec/oferta-
Intent: carreras_UG
Similitud: 0.539
Entidades: {'fechas': [], 'cedula': [], 'carreras': [], 'terminos_dominio': []}
```

`UMBRAL_CONFIANZA_DEFECTO = 0.3`

```
Tú: cuales son los requisitos para la admision

Bot: Lo siento, no entendí tu consulta. ¿Puedes reformularla?
Puedes preguntarme, por ejemplo, sobre:
1. requisitos de admision
2. creacion de cuenta
3. fechas de registro y matricula
4. curso de nivelacion
5. calificaciones, cupos, carreras o documentos necesarios.
Intent: None
Similitud: 0.65
Entidades: {'fechas': [], 'cedula': [], 'carreras': [], 'terminos_dominio': []}
```

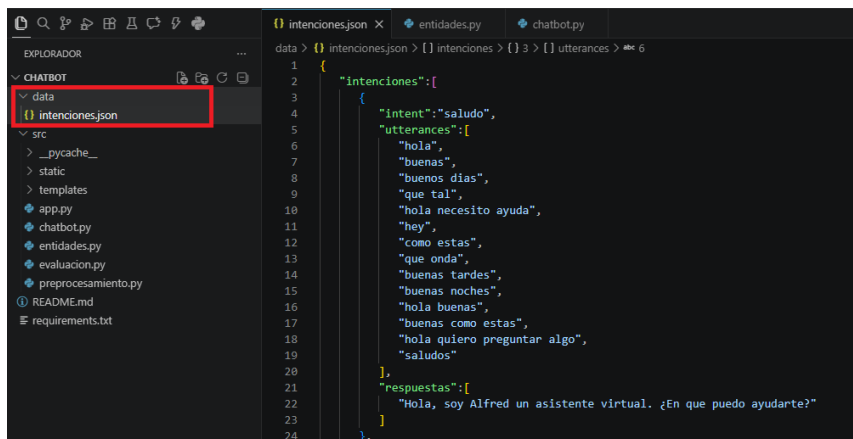
`UMBRAL_CONFIANZA_DEFECTO = 0.7`


```
Tú: cuales son los requisitos para la admision
Bot: Tu duda sobre admision seguramente es sobre los requisitos generales de admision, puedes encontrar esa informacion publicada en el portal oficial https://admission.ug.edu.ec/admission/.
Debes crear una cuenta de aspirante y presentar la documentacion solicitada en el cronograma vigente.
Tambien puedes consultar tu duda en las paginas oficiales de la UG que puedes encontrar solo buscando en cualquier red social contenido sobre la UG.
Intent: preguntar_requisitos_admision
Similitud: 0.65
Entidades: {'fechas': [], 'cedula': [], 'carreras': [], 'terminos_dominio': []}

UMBRAL_CONFIANZA_DEFECTO = 0.5
```

e) Datos (JSON) separados del código

Las intenciones, utterances y respuestas se guardan en data/intenciones.json, completamente separado de la lógica en los archivos .py. Esto permite agregar, quitar o corregir una intención (o simplemente cambiar el texto de una respuesta) sin tocar ni volver a revisar el código del agente, lo cual reduce el riesgo de introducir errores de programación al hacer un cambio de contenido. También hace que el archivo de datos pueda ser editado por alguien que no programe, y que el mismo código (chatbot.py) pudiera reutilizarse para otro dominio simplemente cambiando el JSON.



f) Manejo de errores con try/except

El método responder() de la clase AgenteConversacional envuelve todo el procesamiento de la consulta en un bloque try/except. Antes de intentar detectar la intención, se revisa si la consulta llegó vacía (if not consulta or not consulta.strip()) y se responde pidiendo que se escriba algo, sin ni siquiera llegar al vectorizador. Cualquier otro error inesperado durante el procesamiento (por ejemplo, un problema al calcular la similitud) es capturado por el except Exception as e, y en vez de que el programa se detenga con un traceback, se devuelve una respuesta genérica de error junto con el detalle en el campo "error" del diccionario. De esta manera, ni una consulta vacía ni un error interno detienen el ciclo de conversación.

```
25 class AgenteConversacional:
90     def responder(self, consulta: str) -> dict:
104         intent, score = self.detectar_intencion(consulta)
105         entidades = extraer_entidades(consulta)
106
107         if intent is None:
108             respuesta = random.choice(self.respuestas_fallback)
109         else:
110             respuesta = random.choice(self.respuestas_por_intent[intent])
111
112         # pequeño uso de las entidades detectadas, para que la
113         # respuesta no sea siempre exactamente igual
114         if entidades.get("carreras"):
115             carrera = entidades["carreras"][0]
116             respuesta += f" Vi que mencionaste la carrera de {carrera}, para dudas especificas de esa carrera revisa tambien el portal oficial."
117
118         if entidades.get("cedula"):
119             ultimo_digito = entidades["cedula"][0][-1]
120             respuesta += f" Segun tu cedula, tu ultimo digito es {ultimo_digito}; revisa el cronograma para ver la fecha que te corresponde."
121
122         return {
123             "respuesta": respuesta,
124             "intent": intent,
125             "similitud": round(score, 3),
126             "entidades": entidades,
127         }
128
129     except Exception as e:
130         # Cualquier error inesperado no debe detener el agente
131         return {
132             "respuesta": "Ocurrio un problema procesando tu consulta, intenta de nuevo.",
133             "intent": None,
134             "similitud": 0.0,
135             "entidades": {},
136             "error": str(e),
137         }
```

Conclusiones

El preprocesamiento (normalización de jerga/errores tipográficos y stemming) tuvo un impacto medible: varias consultas con errores de tecleo que antes fallaban, después de agregar el diccionario de normalización pasaron a detectarse correctamente.

La principal limitación observada es la separación entre preguntas dentro y fuera de dominio: al no existir una intención "negativa" explícita bien entrenada, algunas preguntas fuera de tema comparten palabras sueltas con el corpus y terminan clasificadas incorrectamente en vez de caer en fallback.

Como trabajo futuro, se podría ampliar aún más el corpus de utterances, afinar el umbral por intención en lugar de uno global, o agregar más ejemplos negativos (preguntas claramente fuera de dominio) para mejorar la separación con el fallback.